

Subqueries in SQL

A subquery, also known as an inner query or nested query, is a query embedded inside another SQL query. Subqueries are used to retrieve data that will be used by the main query as a condition to further restrict the data to be retrieved.

Key Characteristics of Subqueries

- **Enclosed in Parentheses:** Subqueries are always enclosed within parentheses.
- **Execution Order:** The inner query executes first, and its result is then used by the outer query.
- **Placement:** Subqueries can be used in the **SELECT**, **FROM**, **WHERE**, **HAVING** clauses, and with **INSERT**, **UPDATE**, and **DELETE** statements.
- **Single Column Output:** For **WHERE** and **HAVING** clauses, a subquery can return only one column, though it can return multiple rows. For **SELECT** and **FROM** clauses, it can return multiple columns.

Types of Subqueries

Subqueries can be categorized based on their dependency on the outer query and the number of rows/columns they return.

1. Single-Row Subqueries

These subqueries return only one row to the outer query. They are typically used with single-row comparison operators such as **=**, **>**, **<**, **>=**, **<=**, **<>**.

Example: Find employees who earn more than the average salary.
`SELECT employee_id, first_name, last_name, salary`

`FROM employees`

`WHERE salary > (SELECT AVG(salary) FROM employees);`

2. Multi-Row Subqueries

These subqueries return multiple rows to the outer query. They are used with multi-row comparison operators like **IN**, **NOT IN**, **ANY**, **ALL**, **EXISTS**, **NOT EXISTS**.

Example: Find employees who work in departments located in 📍 Place.
SELECT
employee_id, first_name, last_name, department_id

FROM employees

WHERE department_id IN (SELECT department_id FROM departments WHERE location_id
= (SELECT location_id FROM locations WHERE city = 'London'));

3. Multi-Column Subqueries

These subqueries return multiple columns to the outer query. They are often used in the **FROM** clause (as derived tables) or in the **WHERE** clause for comparisons that involve multiple columns.

Example: Find employees who have the same job ID and salary as employee with ID 100.
SELECT employee_id, first_name, last_name, job_id, salary

FROM employees

WHERE (job_id, salary) = (SELECT job_id, salary FROM employees WHERE employee_id = 100);

4. Correlated Subqueries

A correlated subquery is a subquery that depends on the outer query for its values. It executes once for each row processed by the outer query.

Example: Find employees who earn more than the average salary in their respective departments.
SELECT employee_id, first_name, last_name, salary, department_id

FROM employees e1

WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE e1.department_id = e2.department_id);

Subqueries in Different SQL Clauses

In the **SELECT** Clause (Scalar Subquery)

A subquery in the **SELECT** clause must return a single row and a single column.

Example: Display employee details along with their department's average salary.

```
SELECT employee_id, first_name, last_name, salary,
```

```
    (SELECT AVG(salary) FROM employees e2 WHERE e1.department_id =  
e2.department_id) AS department_avg_salary
```

```
FROM employees e1;
```

In the **FROM** Clause (Derived Table)

A subquery in the **FROM** clause is treated as a temporary table (derived table) and must be given an alias.

Example: Find the maximum salary from each department.

```
SELECT d.department_name, ds.max_salary
```

```
FROM departments d
```

```
JOIN (SELECT department_id, MAX(salary) AS max_salary FROM employees GROUP BY  
department_id) ds
```

```
ON d.department_id = ds.department_id;
```

In the **WHERE** Clause

This is the most common use of subqueries, as demonstrated in the examples above.

Example: Get the names of departments that have at least one employee with a salary greater than 10000.

```
SELECT department_name
```

```
FROM departments
```

```
WHERE department_id IN (SELECT department_id FROM employees WHERE salary >  
10000);
```

With **INSERT** Statement

Subqueries can be used to provide data for an **INSERT** statement.

Example: Insert employees from one department into a new table.

```
INSERT INTO new_employees (employee_id, first_name, last_name, salary,  
department_id)
```

```
SELECT employee_id, first_name, last_name, salary, department_id
```

```
FROM employees
```

```
WHERE department_id = (SELECT department_id FROM departments WHERE  
department_name = 'Sales');
```

With UPDATE Statement

Subqueries can be used to set values in an UPDATE statement.

Example: Increase the salary of employees in the 'IT' department by 10%.
UPDATE employees

```
SET salary = salary * 1.10
```

```
WHERE department_id = (SELECT department_id FROM departments WHERE  
department_name = 'IT');
```

With DELETE Statement

Subqueries can be used to specify rows to be deleted.

Example: Delete employees who do not belong to any department.
DELETE FROM employees

```
WHERE department_id NOT IN (SELECT department_id FROM departments);
```